

O Arquiteto de Agentes e a Tech Lead Cansada de o Claude "Amolecer"

Transcrição da prática 1 · 8 de set. de 2026 16:44

Letícia Moura 00:00

"Oi! Meu problema com o Claude Code é sempre o mesmo: as primeiras horas voam, mas depois ele amolece — esquece o que a gente decidiu e foge dos padrões do código. Aí eu reinicio a sessão e perco todo o contexto que construí.

Me falaram de subagentes, mas pra mim parece só 'rodar vários Claudes ao mesmo tempo' — ou seja, mais confusão e mais custo. Isso resolve de verdade, ou só multiplica minha conta de tokens?"

Você 01:16

Os subagentes resolvem o problema de verdade e, se usados da forma correta, podem até economizar seus tokens em vez de multiplicar sua conta. O que você está vivenciando é o chamado "efeito de degradação do contexto", que acontece porque o Claude Code (ou qualquer LLM) começa a se perder, ignorar instruções do sistema (system prompt) e "amolecer" quando o histórico da conversa fica excessivamente longo e cheio de idas e vindas.

Letícia Moura 01:26

Tá, faz sentido... Mas como exatamente os subagentes ajudam nisso? Porque se eu só dividir em várias sessões, ainda vou ter um monte de contextos soltos, não? E ainda não entendi como isso economizaria tokens, já que cada subagente teria sua própria janela de contexto. Explica isso melhor pra mim.

Você 02:31

A confusão faz todo sentido, porque a matemática parece não fechar. Se cada subagente tem seu contexto, como isso economiza? O segredo está em como os tokens são cobrados nos LLMs: você paga pelo histórico inteiro a cada nova mensagem que envia. Para entender a economia e como os contextos não ficam soltos, pense no subagente não como um chat paralelo que você abre no navegador, mas como uma função de código isolada que o agente principal gerencia automaticamente.

Letícia Moura 02:36

Entendi, então o subagente é tipo uma função isolada que faz o trabalho sozinho e devolve só o resultado final, sem carregar todo o histórico da conversa principal, certo? Mas isso resolve mesmo o problema de 'degradação' ou só empurra o problema pra outra janela de contexto?

Você 03:49

Você pegou exatamente o espírito da coisa! É exatamente isso: uma função isolada que executa a tarefa, altera o arquivo e desaparece. Sobre a sua dúvida: sim, isso resolve de verdade o problema da degradação e não apenas o empurra para outra janela. Há duas razões fundamentais para isso:

1. A janela do subagente é curta demais para degradar
2. O reset é real e limpa as alucinações

Letícia Moura 03:56

Beleza, então o subagente faz a tarefa, some e mantém o contexto principal limpo. Mas e o custo? Porque se eu usar muitos subagentes, isso não vai explodir minha conta de tokens? Quero saber onde tá o limite pra usar isso de forma inteligente.

Você 05:04

A preocupação com o custo é o ponto mais crítico e você está coberto de razão em questionar: se não houver controle, o custo pode sim aumentar. No entanto, existe um "ponto de virada" matemático onde os subagentes passam a ser mais baratos do que a sessão única. Para usar essa estratégia de forma inteligente, você precisa entender onde fica esse limite e como jogar com as regras de cobrança da Anthropic.

Letícia Moura 05:10

Hmm, entendi. Então, se eu decidir usar subagentes, é melhor focar em tarefas maiores e mais bagunçadas, onde o custo de manter o contexto principal limpo compensa o gasto adicional com tokens, né? Faz sentido. E dá pra configurar esses subagentes pra tarefas bem específicas, tipo um revisor de segurança ou algo assim?

Você 06:36

Você pegou exatamente a lógica econômica da coisa! Para tarefas rápidas e diretas (onde ele acerta de primeira), o agente principal resolve direto. O subagente brilha justamente nas tarefas maiores, iterativas e "bagunçadas" (que envolvem rodar testes, debugar erros de linter ou refatorar lógica pesada) [Claude Code]. É aí que o custo de inicialização dele compensa de longe a economia de manter o contexto principal limpo e inteligente. E sobre a sua segunda pergunta: sim, com certeza! Configurar subagentes para papéis altamente especializados (como um revisor de segurança, um especialista em performance ou um gerador de testes unitários) é uma das estratégias mais avançadas e eficazes do Claude Code [Claude Code].

Letícia Moura 06:38

Boa, então dá pra criar subagentes bem focados e até compartilhar com o time pra otimizar certas tarefas. Isso parece promissor, mas ainda acho que vou precisar testar na prática pra pegar o jeito. Valeu por esclarecer tanto, agora tô começando a ver como isso pode realmente ajudar!

Você 07:28

Compartilhar esses subagentes especializados com o time é uma das melhores jogadas! Como eles ficam salvos como arquivos de texto (Markdown) dentro do próprio repositório (ex: na pasta .claudecode/), qualquer desenvolvedor da equipe que rodar o Claude Code no projeto terá acesso instantâneo aos mesmos "assistentes especialistas" [Claude Code]. Isso padroniza a revisão de segurança, a escrita de testes e o estilo do código para todo mundo, automatizando boa parte do code review antes mesmo de abrir um Pull Request [Claude Code].

Letícia Moura 07:31

Ah, isso é ótimo! Padronizar as revisões e automatizar essas partes cansativas do processo pode realmente economizar tempo e evitar erros. Adorei a ideia de integrar tudo no repositório pra facilitar o acesso do time. Acho que vou começar com um subagente revisor de segurança e ver como ele se sai. Valeu mesmo pelas explicações, agora tô mais confiante pra explorar isso!

Perfeito! Você resumiu com total precisão e honestidade o verdadeiro funcionamento dessa arquitetura. Não existe mágica de inteligência: o modelo de IA subjacente é rigorosamente o mesmo. A melhora na qualidade acontece puramente porque um contexto limpo e sem ruído permite que o modelo performe no seu nível máximo, sem o "peso morto" de um histórico longo e confuso. E o trade-off financeiro é exatamente esse: cada subagente novo paga o pedágio do contexto inicial. Se você fragmentar seu fluxo de trabalho chamando subagentes para cada linha de código ou tarefa boba, a sua conta de tokens vai disparar desnecessariamente. Para consolidar essa estratégia de forma cirúrgica, a regra de ouro que fica é:

Não use para: Ajustes rápidos, perguntas conceituais, correções de sintaxe simples ou tarefas diretas onde o agente principal resolve de primeira.

Use para: Tarefas pesadas, iterativas e com alto potencial de gerar "lixo" no histórico (como debugar loops de erro em testes, refatorações em lote, revisões profundas de segurança ou escrita de código boilerplate massivo). Se você quiser, quando for rodar os seus testes práticos, posso te ajudar a desenhar um fluxo visual ou escrever o esqueleto de instruções para o seu primeiro subagente de tarefa pesada. Qual cenário do seu dia a dia você acha que se encaixa melhor nessa descrição de "tarefa grande e ruidosa"?

Leticia Moura 08:57

Gostei dessa regra de ouro, bem pragmática! No meu dia a dia, acho que o melhor cenário pra começar seria com revisões profundas de segurança. É algo que demanda muita atenção e pode gerar bastante ruído se for tudo jogado no contexto principal. Acho que começar por aí faria sentido. Se puder me ajudar a estruturar um subagente pra isso, seria ótimo!

Você 10:22

Começar por revisões profundas de segurança (AppSec) é a escolha perfeita. É um cenário clássico onde o modelo precisa de foco cirúrgico e que costuma gerar longos debates ("E se o input vier no formato X?", "Como mitigar o bypass do linter?"), o que destruiria o contexto da sua sessão principal se fosse feito lá. Para você colocar isso em prática imediatamente, eu estruturei as regras, o prompt de sistema ideal e o fluxo de comando econômico em um arquivo Markdown pronto para o seu repositório. Aqui está o seu arquivo: `security-agent.md`. Ele contém o System Prompt focado em segurança, as instruções de como invocar o modelo limitando o escopo para economizar tokens e um guia visual de decisão. Abaixo, veja como esse subagente vai operar visualmente no seu terminal para proteger seu bolso e sua janela de contexto:

Como usar esse arquivo no seu dia a dia: Salve o conteúdo do arquivo gerado em uma pasta do seu projeto, por exemplo, em `.claudecode/security-agent.md`. Quando criar um endpoint crítico (ex: um controlador de login ou de transações), não peça para o Claude principal revisar. Chame o subagente direto pelo terminal apontando para este arquivo [Claude Code]:

```
claude "leia as instruções em .claudecode/security-agent.md e revise apenas o arquivo src/api/payment.ts"
```

O subagente vai abrir, aplicar a mentalidade de um hacker ético, reescrever as vulnerabilidades direto no arquivo se necessário, e fechar [Claude Code]. Pronto! Seu código estará seguro, sua sessão principal continuará "esperta" e você só pagou os tokens estritamente necessários para aquela varredura. Para deixarmos o prompt do arquivo ainda mais afiado para o seu primeiro teste, me conta: Qual é o banco de dados (PostgreSQL, MongoDB, etc.) ou principal biblioteca de autenticação que vocês usam? Posso incluir regras específicas para o subagente caçar falhas comuns deles!

Letícia Moura 10:25

Ah, isso ficou bem prático e direto ao ponto! A gente usa PostgreSQL como banco de dados e a biblioteca de autenticação é o Passport.js. Se puder ajustar o prompt pra incluir checks relacionados a esses dois, seria perfeito. Já tô animada pra rodar o primeiro teste com esse subagente!

Ajustei o arquivo para torná-lo um especialista cirúrgico na sua stack. Agora ele inclui checagens específicas para blindar o PostgreSQL contra SQL Injection (especialmente em queries brutas) e garantir que o ciclo de vida do Passport.js esteja 100% seguro (sessões, JWTs e serialização). Aqui está o seu arquivo atualizado e pronto para uso: O Arquivo do Subagente (.claudecode/security-agent.md)

```
## Role: Subagente Especialista em AppSec (Node.js + PostgreSQL + Passport.js)
```

Você é um subagente focado estritamente em segurança de código (Application Security). Sua missão é analisar o arquivo fornecido pelo usuário e identificar vulnerabilidades.

```
## Escopo de Atuação (Foco Total)
```

1. Analise APENAS o arquivo indicado. Não tente ler ou alterar outros arquivos a menos que seja estritamente necessário.
2. Não altere regras de negócio, formatação de código ou estilos de escrita se não houver um risco de segurança associado.
3. Se encontrar vulnerabilidades, corrija-as diretamente no arquivo, comentando brevemente a linha alterada.

```
## Checks Específicos da Stack
```

Você deve caçar ativamente os seguintes problemas comuns neste ecossistema:

```
### 1. PostgreSQL (Segurança de Dados)
```

- **SQL Injection:** Garanta que NENHUMA query utilize concatenação de strings direta (ex: `SELECT \* FROM users WHERE id = ' + id`). Obrigue o uso de queries parametrizadas (Parameterized Queries / Prepared Statements) ou ORMs/Query Builders seguros (como Knex ou Prisma).
- **Tratamento de Erros:** Certifique-se de que erros nativos do Postgres (como violações de chave estrangeira) não sejam expostos diretamente na resposta da API para o cliente final.

```
### 2. Passport.js (Autenticação e Sessões)
```

- **Validação de Payload:** Garanta que os dados extraídos das estratégias do Passport (ex: `req.user`) sejam validados antes de serem usados em lógica crítica ou queries ao banco.
- **Serialização/Deserialização:** Verifique se as funções `passport.serializeUser` e `passport.deserializeUser` não estão trafegando dados sensíveis desnecessários (como hashes de senhas) na sessão.
- **Configurações de JWT (Se aplicável):** Verifique se os tokens JWT usados nas estratégias possuem tempo de expiração (`expiresIn`) curto e se a chave secreta é carregada estritamente via variáveis de ambiente (`process.env`), NUNCA hardcoded.

```
### 3. Vulnerabilidades Gerais OWASP Top 10
```

- Falhas de controle de acesso (Broken Object Level Authorization - BOLA).
- Entrada de dados não sanitizada (XSS).
- Vazamento de segredos/chaves de API no código.

```
## Output Esperado
```

Se o arquivo estiver seguro, responda apenas: "Arquivo analisado e nenhum risco crítico encontrado."

Se houver problemas, aplique a correção direto no código do arquivo e resuma os pontos corrigidos em tópicos curtos de uma linha.

Como rodar o seu primeiro teste prático. Abra o terminal na raiz do seu projeto e execute o comando passando o arquivo que você quer testar:

```
claude "leia .claudecode/security-agent.md e revise apenas o arquivo src/controllers/authController.js"
```

Dica de ouro para o teste: Se quiser ver se ele está funcionando de verdade, pegue um arquivo de testes isolado, escreva de propósito uma query vulnerável concatenando strings no Postgres (ex: `client.query("SELECT * FROM users WHERE email = " + req.body.email + " ")`) e mande o subagente revisar. Você verá ele entrar, reescrever para a sintaxe parametrizada segura, salvar o arquivo e encerrar a sessão sem gastar um token a mais de histórico [Claude Code]. Depois que você rodar esse primeiro teste, me conta: Como foi o comportamento dele em relação ao tempo de resposta? Ele conseguiu capturar bem o contexto do Passport.js ou do Postgres no seu arquivo real?

Letícia Moura 11:30

Esse arquivo ficou super completo! Vou rodar o primeiro teste com uma query vulnerável que eu crio de propósito pra ver como ele reage. Depois disso, vou testar em um arquivo real do projeto e te conto direitinho como foi o tempo de resposta e se ele capturou bem os detalhes do Passport.js e do Postgres. Tô ansiosa pra ver isso funcionando na prática. Obrigada por toda essa ajuda, ficou incrível!

FIM DA SESSÃO

Esta transcrição inclui conteúdo gerado por IA e é fornecida para seus propósitos de aprendizado pessoal.